

GutBrain Task 6

Challenge:

NER & RE models

ADVANCED TEXT ANALYTICS

Prof. Giorgio Maria di Nunzio

SAMUEL PIRON

10/04/2025

A large red circle is positioned on the left side of the slide, partially cut off by the edge.

OVERVIEW

Introduction

Data Overview

NER Model

Data Augmentation

RE Model

Model Evaluation

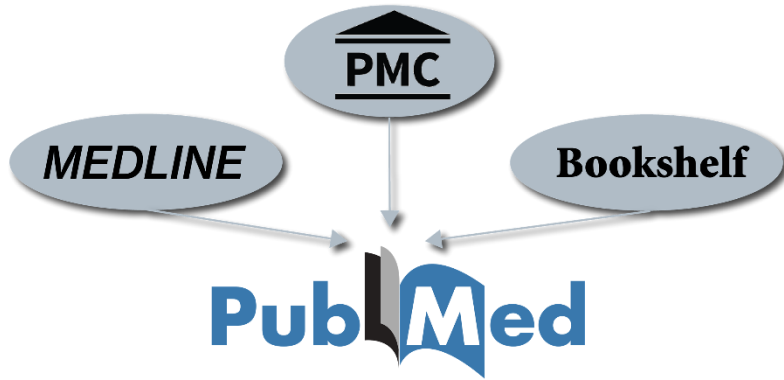
Conclusion and Future Work

A decorative element on the left side of the slide consisting of four vertical bars of varying heights and widths, all in a dark purple color.

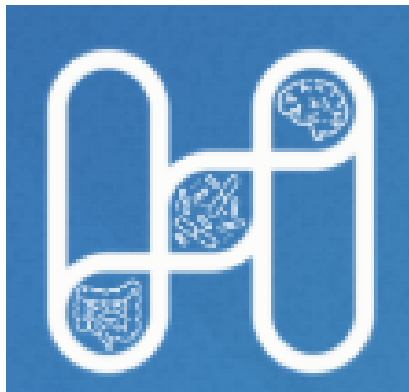
I. Introduction



Introduction



- **PubMed** is a popular website that offers a searchable database of citations and abstracts of biomedical and life sciences literature.
- It comprises more than **38 million** citations and abstracts.
- From **1996**, it has been maintained by the National Center for Biotechnology Information (**NCBI**).



Introduction (II)

The **Gut Brain Challenge** aims to foster the development of **Information Extraction (IE)** systems that support experts by automatically extracting and linking knowledge from scientific literature.

The challenge is divided into two main subtasks:

- **NER subtask:** from PubMed abstracts, the participants are asked to extract named entities about the gut-brain interplay.
- **RE subtask:** the participants are asked to identify binary and ternary relations between any pair of entities they extract within an abstract.

Samuel Piron

A decorative element on the left side of the slide consisting of four vertical bars of increasing height from left to right, all in a dark purple color.

2. Data Overview

A horizontal dark purple line spanning the width of the slide near the bottom.

Gut Brain Dataset

The Gut Brain Dataset is composed of **four** Train collections and **one** Dev Collection:

- Platinum Collection
- Gold Collection
- Silver Collection
- Bronze Collection
- Dev Dataset

Each collection is composed of NER-tagged entities with the label, text span, start index, and end index.

It is composed of relations between the NER-tagged entities, in particular binary tag based, ternary tag based, and ternary mention based.

```
"38068763": {  
  "metadata": {  
    "title": "Analysis of the Efficacy of Diet and Short-T  
    "author": "Natalia Komorniak; Mariusz Kaczmarczyk; Igo  
    "journal": "Nutrients",  
    "year": "2023",  
    "abstract": "(1) Background: studies have shown that s  
    "annotator": "expert_3"  
  },  
  "entities": [  
    {  
      "start_idx": 74,  
      "end_idx": 92,  
      "location": "title",  
      "text_span": "Depressive Symptoms",  
      "label": "DDF"  
    },  
    {  
      "start_idx": 97,  
      "end_idx": 104,  
      "location": "title",  
      "text_span": "Patients",  
      "label": "human"  
    },  
    {  
      "start_idx": 45,  
      "end_idx": 52,  
      "location": "abstract",  
      "text_span": "patients",  
      "label": "human"  
    }  
  ]  
}
```

A decorative element on the left side of the slide consisting of four vertical bars of increasing height from left to right, all in a dark purple color.

3. NER Model



NER Model – GLiNER (I)

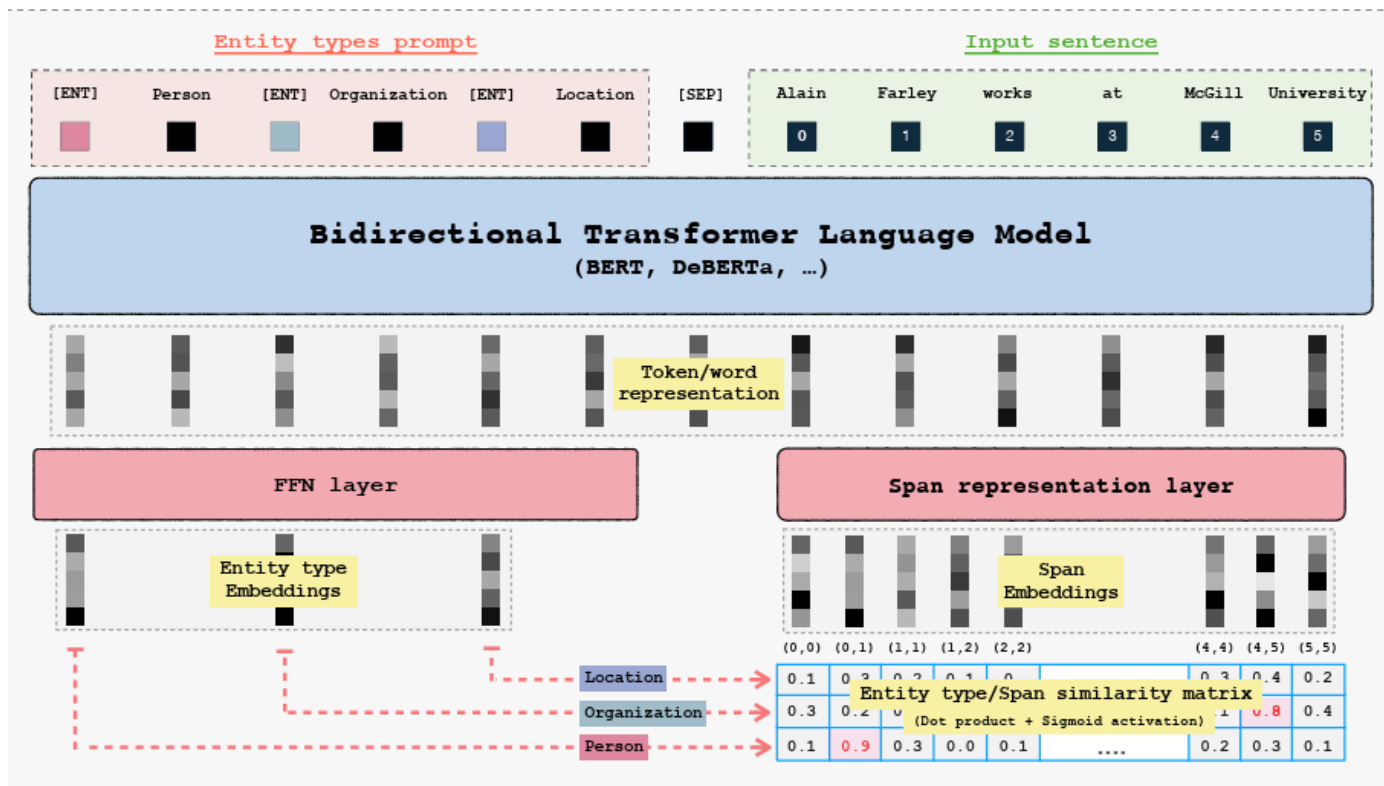
GLiNER is a Named Entity Recognition (**NER**) model capable of identifying any entity type using a **Bidirectional transformer** encoder (BERT-like).

It provides a practical alternative to traditional NER models, which are limited to predefined entities, and Large Language Models (LLMs) that, despite their flexibility, are costly and large for resource-constrained scenarios.

GLiNER paper can be found here:

<https://arxiv.org/abs/2311.08526>

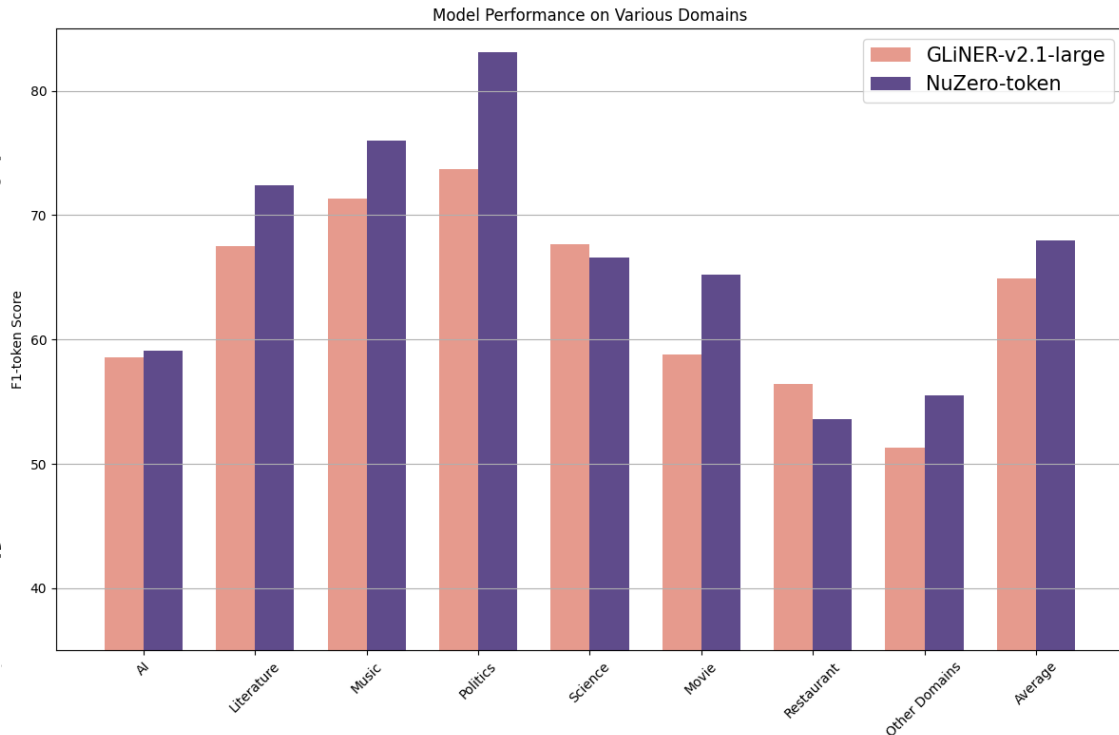
NER Model – GliNER (II)



NER Model – GLiNER (III)

The **best model** found to have a better performance in terms of training loss is the **numind/NuNER_Zero**.

- It is a zero-shot NER Model. It uses GLiNER architecture: its input are a concatenation of entity types and text.
- It is a token classifier that allows the detection of arbitrarily long entities.
- It is trained on numind/NuNER-v2.0



NER Model - Implementation

The collections used to train the **NER** model:

- Platinum
- Gold

The silver dataset was used with both platinum and gold, but there were no improvements.

Tokenizer

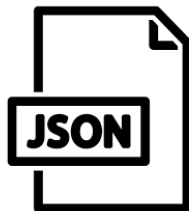
The **Tokenizer** that was used during the data processing for the **GLiNER** model with the merged dataset is the [microsoft/deberta-v3-large](#).

The **DeBERTaV3** large model (based on BERT) comes with 24 layers and a hidden size of 1024. It has 304M backbone parameters with a vocabulary containing 128K tokens, which introduces 131M parameters in the Embedding layer.

Samuel Piron

NER Model – Train Pipeline (I)

1) **Load Data:** load the annotated dataset containing titles, abstracts and annotated entities.



Train merged.json

2) **Initialize Tokenizer:** Tokenizer breaks down text into tokens, making it ready for the model input.



DE-BERTa-v3-large

3) **Preprocess Data:** combine titles and abstracts, convert character level entity annotations into token level spans using tokenizer's offset mapping.

"This is the first step in the NLP pipeline"

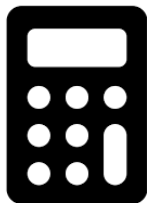
Tokenizer

'This'	'is'	'the'	'first'	'step'	'in'	'the'	'NLP'	'pipeline'
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

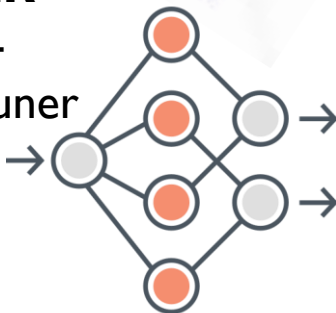
NER Model – Train Pipeline (II)

4) Define Hyperparameters:

Set parameters such as batch size, learning rate, num. of training steps, warmup ratio and others for training stability.



5) **Model Setup:** initialize **NER** model using GLiNER with pre-trained weights (numind/NuNuner Zero) and configure sampling parameters to handle training complexity.



6) **Training Loops:** iterate through batches, perform forward pass, compute loss and apply back prop. Update model parameters using **AdamW** optimizer.



7) **Evaluation:** periodically evaluate the model performance using validation data and produce Micro and Macro, F1-score, entity-level accuracy and save it.



Samuel Piron

NER Model – Train Pipeline (III)

```
def preprocess_data(articles):
    processed_data = []
    for article_id, article_info in articles.items():
        # Extract title and abstract from the metadata.
        title_text = article_info["metadata"]["title"]
        abstract_text = article_info["metadata"]["abstract"]

        # Combine title and abstract with a space separator.
        combined_text = title_text + " " + abstract_text

        # Tokenize the combined text with offset mappings (do not add special tokens)
        encoding = tokenizer(combined_text, return_offsets_mapping=True, add_special_tokens=False)
        tokenized_text = tokenizer.convert_ids_to_tokens(encoding["input_ids"])
        offset_mapping = encoding["offset_mapping"]

        new_ner = []
        # Process each entity from either title or abstract.
        for ent in article_info["entities"]:
            # Only consider entities from the title or abstract.
            if ent["location"] not in ["title", "abstract"]:
                continue

            start_char = ent["start_idx"]
            end_char = ent["end_idx"]
            label = ent["label"].lower()
```

Data Processing function.

```
def train(model, config, train_data, eval_data=None):
    model = model.to(config.device)

    # Set sampling parameters.
    model.set_sampling_params(
        max_types=config.max_types,
        shuffle_types=config.shuffle_types,
        random_drop=config.random_drop,
        max_neg_type_ratio=config.max_neg_type_ratio,
        max_len=config.max_len
    )

    model.train()
    train_loader = model.create_dataloader(train_data, batch_size=config.batch_size, shuffle=True)
    optimizer = model.get_optimizer(config.lr_encoder, config.lr_others, config.optimizer)
    pbar = tqdm(range(config.num_steps))
    num_warmup_steps = int(config.num_steps * config.warmup_ratio) if config.warmup_ratio > 0 else 0
    scheduler = get_cosine_schedule_with_warmup(
        optimizer,
        num_warmup_steps=num_warmup_steps,
        num_training_steps=config.num_steps
    )
```

Train function.

Four vertical bars of increasing height from left to right, colored in a dark purple shade.

4. Data Augmentation



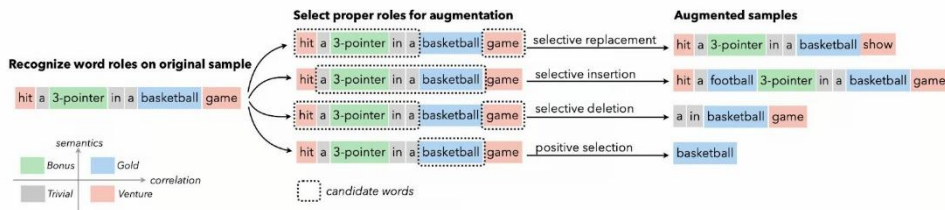
Data Augmentation

Data augmentation is a technique of artificially **increasing** the **training set** by creating modified copies of a dataset using existing data.

It includes making minor changes to the dataset or using deep learning to generate new data points.

Why should we use data augmentation?

- It helps machine learning models perform better by improving the existing data.
- It prevents *overfitting*, improves *accuracy*, and creates *diversity* in training data.



Data Augmentation - Implementation (I)

1) Retrieve from **MeSH** (the Official Medicine Vocabulary Online) all the terms and their synonyms.

2) Parse the .xml file obtained, and after that, save it into a JSON format.

3) For each term and its synonyms, compute the embedding using the model all-mpnet-base-v2.



```
dtype=float32), 'syn': {'pregnancy after 35': array([ 4.63531353e-03,
 2.97347251e-02, -3.92423943e-03, -7.24278837e-02,  3.84213440e-02,
-4.32130583e-02,  8.73802528e-02,  5.42837717e-02, -2.92770416e-02,
-7.29111908e-03, -1.80010162e-02,  7.09951296e-02,  1.48122245e-02,
 6.44895509e-02, -1.10682905e-01,  6.54023811e-02, -4.95651104e-02,
-5.68322912e-02, -2.56238282e-02,  7.21648149e-03,  2.92945001e-02,
 1.10893629e-01,  6.99814782e-02, -7.91331679e-02,  3.35495509e-02,
-2.34043133e-02, -7.48873129e-02, -3.99700701e-02, -2.55217571e-02,
 6.37074932e-02, -7.13312104e-02, -8.36416855e-02, -7.44862109e-02,
-1.17245436e-01,  5.79124764e-02, -1.49343833e-02,  6.90168068e-02,
 6.23305514e-03, -9.88510810e-03, -1.40163183e-01,  5.96894883e-02,
-4.56312001e-02, -1.70112345e-02,  3.87386680e-02, -1.30609274e-02,
-1.90430954e-02,  3.08640730e-02, -4.76355739e-02, -2.88699451e-03,
```

Data Augmentation - Implementation (II)

- 4) Compute the **Cosine Similarity** between the original term and its synonyms. If the similarity is above a threshold, it is kept, otherwise, it is rejected.
- 5) Replace in the title or in the abstract the original term with the augmented one and save the new file.
- 6) With a trained model, use it to predict the entities and then retrain it using the augmented file.



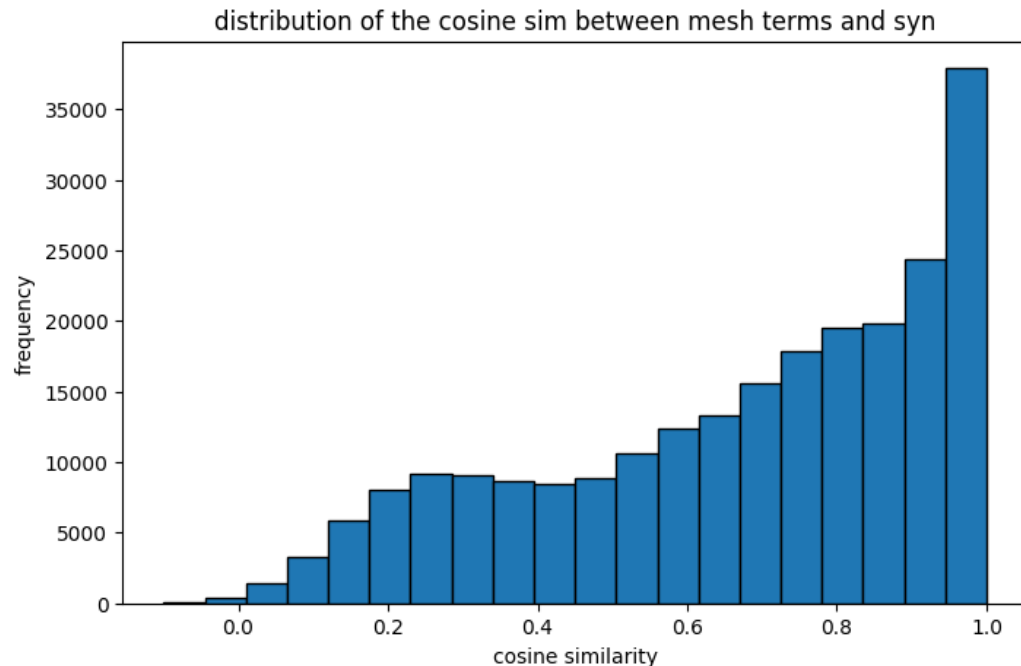
```
dtype=float32), 'syn': {'pregnancy after 35': array([[ 4.63531353e-03,
 2.97347251e-02, -3.92423943e-03, -7.24278837e-02,  3.84213440e-02,
-4.32130583e-02,  8.73802528e-02,  5.42837717e-02, -2.92770416e-02,
-7.29111908e-03, -1.80010162e-02,  7.09951296e-02,  1.48122245e-02,
 6.44895509e-02, -1.10682905e-01,  6.54023811e-02, -4.95651104e-02,
-5.68322912e-02, -2.56238282e-02,  7.21648149e-03,  2.92945001e-02,
 1.10893629e-01,  6.99814782e-02, -7.91331679e-02,  3.35495509e-02,
-2.34043133e-02, -7.48873129e-02, -3.99700701e-02, -2.55217571e-02,
 6.37074932e-02, -7.13312104e-02, -8.36416855e-02, -7.44862109e-02,
-1.17245436e-01,  5.79124764e-02, -1.49343833e-02,  6.90168068e-02,
 6.23305514e-03, -9.88510810e-03, -1.40163183e-01,  5.96894883e-02,
-4.56312001e-02, -1.70112345e-02,  3.87386680e-02, -1.30609274e-02,
-1.90430954e-02,  3.08640730e-02, -4.76355739e-02, -2.88699451e-03,
```

Data Augmentation - Implementation (II)

The data augmentation process does not improve so much the P, R, F1 score.

Why?

The GLiNER model maps similar words close so that most of the terms with the Cosine similarity greater than 0.8 that are replaced do not affect the precision, recall and f1 score so much.



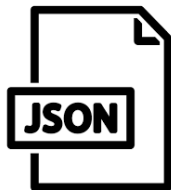
A decorative element on the left side of the slide consisting of four vertical bars of increasing height from left to right, all in a dark purple color.

5. RE Model

RE Model – Implementation (I)

1) Load the JSON file

& Tokenizer: load the articles, relations and entities; then use BERT Tokenizer to prepare the input for the NN adding [E1],[E2] to highlight entities in the text.



2) Data Preprocessing:

Separate entities in the title or in the abstract, filter relations that involve both of them; create valid relations and non valid relations.



3) **The PubMedBERT Model:** initialize a BERT model pre-trained on PubMed abstracts. Resize the token embedding to include the markers ([E1], etc.). Set up the model to perform Sequence Classification: each sequence corresponds to a potential relation between two entities.



4) **Training Setup:** Set optimizer to AdamW; set up hyperparameters that are going to be used to train the model (epoch, steps, learning rate).



Samuel Piron

RE Model – Implementation (II)

5) **Model Training:** the model undergoes forward and backward pass. Update the scheduler and optimizer at each step.



6) **Evaluation Metrics:** after each epoch, evaluate the model on a validation set printing Precision, Recall, F1 Score.



7) **Save the model:** save the model weights and tokenizer for future use.



RE Model – Train Pipeline (III)

```
def _process_data(self, data):

    for article in tqdm(data.values(), desc="Processing articles"):
        title_text = article["metadata"]["title"]
        abstract_text = article["metadata"]["abstract"]

        # Separate entities into those for 'title' vs. 'abstract'
        title_entities = []
        abstract_entities = []
        for ent in article["entities"]:
            if ent["location"] == "title":
                # Offsets in the title remain as-is
                e = {
                    "start": ent["start_idx"],
                    "end": ent["end_idx"],
                    "text": ent["text_span"],
                    "label": ent["label"]
                }
                title_entities.append(e)

def _process_relations(self, text, entities, relations):
    samples = []
    entity_map = {(e["start"], e["end"]): e for e in entities}

    for rel in relations:
        subj = entity_map.get((rel["subject_start_idx"], rel["subject_end_idx"]))
        obj = entity_map.get((rel["object_start_idx"], rel["object_end_idx"]))

        if subj and obj:
            sample = self._create_sample(text, subj, obj, rel["predicate"])
            if sample:
                samples.append(sample)
            # If a new predicate label wasn't in label2id initially, add it
            if rel["predicate"] not in self.label2id:
                self.label2id[rel["predicate"]] = len(self.label2id)
```

```
def train():
    # Load data
    with open("/kaggle/input/train-tratinum-json/train_platinum.json") as f:
        data = json.load(f)

    # Initialize tokenizer with special markers
    tokenizer = BertTokenizer.from_pretrained(MODEL_NAME)
    tokenizer.add_tokens(MARKERS, special_tokens=True)

    # Create dataset
    dataset = RelationExtractionDataset(data, tokenizer)

    # Prepare stratification mask
    label_counts = Counter()
    for sample in dataset.samples:
        label_counts[sample["label"]] += 1

    stratify_mask = []
    for sample in dataset.samples:
        label = sample["label"]
        if label_counts[label] >= 2:
            stratify_mask.append(label)
        else:
            stratify_mask.append("##RARE##")
```

Process data and relations function and train function.

RE Model – Key Points

1) Using a pre-trained model on PubMed abstracts improved the precision, recall and f1 score of the predictions.

The model uses special entity markers to improve its ability to recognize relationships between entities.

2) **Label2id** converts relationship labels from text to numerical form, essential for multi-class classification.

It allows loss calculation and **efficient** model training, **simplifies** predictions, calculates loss, and interprets model predictions easily.

3) The controlled negative ratio helps to create negative samples: it helps the model to distinguish between actual relationships and non-relationships.

A decorative element on the left side of the slide consisting of four vertical bars of increasing height from left to right, all in a dark purple color.

6. Model Evaluation



Evaluation Metrics

- **TP (True Positives)** refers to the number of correctly predicted entities or relations.
- **FP (False Positives)** refers to the number of wrongly predicted entities or relations.
- **FN (False Negatives)** refers to the number of entities or relations in the ground truth that were not predicted.
- **L** is the set of labels referring to each subtask (e.g. for NER entity labels; for Binary subject, object)
- Micro-average Precision: $P_{\text{micro}_{\text{avg}}} = \frac{\sum_{l \in \mathcal{L}} TP_l}{\sum_{l \in \mathcal{L}} TP_l + FP_l}$
- Micro-average Recall: $R_{\text{micro}_{\text{avg}}} = \frac{\sum_{l \in \mathcal{L}} TP_l}{\sum_{l \in \mathcal{L}} TP_l + FN_l}$
- Micro-average F1-score: $F1_{\text{micro}_{\text{avg}}} = 2 \times \frac{P_{\text{micro}_{\text{avg}}} \times R_{\text{micro}_{\text{avg}}}}{P_{\text{micro}_{\text{avg}}} + R_{\text{micro}_{\text{avg}}}}$

NER Model - Evaluation

Test Train	Platinum Collection	Gold Collection	Silver Collection	Bronze Collection
Platinum + Gold	P:0.9634 R:0.8906 F1:0.9256	P:0.9775 R:0.8775 F1:0.9248	P:0.7862 R:0.6708 F1:0.7239	P:0.9021 R:0.7905 F1:0.8426
Platinum + Gold + Silver	P:0.8179 R:0.8233 F1:0.8205	P:0.8325 R:0.8386 F1:0.8355	P:0.8074 R:0.7799 F1:0.7934	P:0.8165 R:0.8179 F1:0.8172
Platinum + Gold + Bronze Augmented	-	-	-	P:0.7071 R:0.8025 F1:0.7518

The **Dev Dataset** in this evaluation is considered as the Test Dataset.

Precision: 0.8017, Recall: 0.6947, F1: 0.7444

The model used for **NER** of the *Dev Dataset* is **platinum + gold**.

Samuel Piron

Binary Tag-Based Relation Extraction

Test Train	Platinum Collection	Gold Collection	Silver Collection	Bronze Collection
Platinum + Gold	P:0.5130 R:0.9135 F1:0.6571	P:0.4154 R:0.8640 F1:0.5610	P:0.4397 R:0.7479 F1:0.5538	P:0.3471 R:0.8646 F1:0.4953

The **Dev Dataset** in this evaluation is considered as the Test Dataset.

Precision: 0.5978

Recall: 0.7364

F1: 0.6599

This is the best result so far, with a threshold of 0.7 for the predictions.

Ternary Tag-Based Relation Extraction

Test Train	Platinum Collection	Gold Collection	Silver Collection	Bronze Collection
Platinum + Gold	P:0.8753 R:0.6757 F1:0.7627	P:0.7694 R:0.6085 F1:0.6795	P:0.6938 R:0.4493 F1:0.5454	P:0.7060 R:0.5433 F1:0.6140

The **Dev Dataset** in this evaluation is considered as the Test Dataset.

Precision: 0.8100

Recall: 0.3522

F1: 0.4909

This is the best result so far, with a threshold of 0.7 for the predictions.

Ternary Mention-Based Relation Extraction

Test Train	Platinum Collection	Gold Collection	Silver Collection	Bronze Collection
Platinum + Gold	P:0.8288 R:1.2329 F1:0.9912	P:0.7283 R:1.0661 F1:0.8654	P:0.5762 R:0.5259 F1:0.5499	P:0.6382 R:0.9184 F1:0.7531

The **Dev Dataset** in this evaluation is considered as the Test Dataset.

Precision: 0.6573

Recall: 0.5821

F1: 0.6174

This is the best result so far, with a threshold of 0.7 for the predictions.

A decorative element on the left side of the slide consisting of four vertical bars of increasing height from left to right, colored in a dark purple shade.

7. Conclusions and Future Work



Conclusions & Future Work

Conclusions

- The training strategies of the NER and RE models helped perform complex information extraction on biomedical data.
- The process of Data Augmentation did not help to improve the scores of the NER model caused by the GLiNER embeddings.
- The results compared to the Baseline provided in the official website show an improvement in the relation extraction subtask.

Future Work

- Try, for example, the SCiBert v2.1-Large for the RE subtask and compare the results with the PubMedBERT model.
- Test the trained model on the test data upon release.

Code, Runs & Devices used for Training

The code with the implementation can be found here:

<https://github.com/Vezzero/ataupd2425-gainer>

The training setups are:

- Tesla T4 GPU Google Colab (<https://colab.research.google.com/>)
- NVIDIA T4 x 2 GPU Kaggle (<https://www.kaggle.com/>)
- 8x NVIDIA A40 GPU DEI Machines (<https://docs.dei.unipd.it/>)

Bibliography

PubMed: <https://pubmed.ncbi.nlm.nih.gov/>

Clef2025: <https://clef2025.clef-initiative.eu/index.php?page=Pages/Labs/BioASQ.html>

Embedding Model: <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

NuNER_Zero: https://huggingface.co/numind/NuNER_Zero

MeSH: <https://www.nlm.nih.gov/mesh/meshhome.html>

PubMedBERT: <https://huggingface.co/microsoft/BiomedNLP-BiomedBERT-base-uncased-abstract-fulltext>

Gut Brain 2025 Challenge
ADVANCED TEXT ANALYTICS
THANK YOU FOR YOUR
ATTENTION

10/04/2025

Any Questions?

Samuel Piron